

FITBUDDY PROJECT

DETAILED TESTING SECTION

Testing Documentation

This section describes the testing methodology, test environment, test cases, validation procedures, non-functional testing, defect handling, regression testing, and final test results for the FitBuddy application.

1. Introduction to Testing

Software testing is the systematic process of evaluating FitBuddy to identify defects and verify that the application satisfies its functional and non-functional requirements. Testing is performed from the user's point of view as well as at individual component and integration levels. The main objective is to ensure that FitBuddy is reliable, secure, usable, responsive, and capable of handling valid and invalid user actions correctly.

Testing also confirms that information entered by users is validated, stored, retrieved, and displayed consistently. Important workflows such as account access, profile management, fitness activities, workout tracking, progress monitoring, navigation, and error handling are included in the testing scope.

2. Objectives of Testing

1. Verify that all major FitBuddy features work according to their requirements.
2. Detect functional defects before deployment.
3. Validate required fields, data formats, ranges, and user inputs.
4. Confirm that user data is stored and retrieved correctly.
5. Verify authentication and access control behaviour.
6. Ensure correct navigation between screens and modules.
7. Check error handling and meaningful user feedback.
8. Evaluate usability, compatibility, responsiveness, and performance.
9. Perform regression testing after corrections or feature changes.
10. Confirm that the final application is stable enough for intended use.

3. Testing Strategy

FitBuddy uses a combination of black-box, requirement-based, positive, negative, boundary-value, and regression testing. Positive testing verifies that valid inputs produce the expected result. Negative testing verifies that invalid or unexpected inputs are safely rejected or handled. Boundary-value testing concentrates on values at the minimum and maximum permitted limits.

Testing Level / Type	Purpose
Unit Testing	Tests individual functions, validators, calculations, UI components, and small modules independently.
Integration Testing	Verifies communication between modules such as authentication, profile, workout, progress, and storage/API components.
System Testing	Tests the complete FitBuddy application as one integrated system.
Functional Testing	Confirms that each feature produces the expected result for defined inputs.
Regression Testing	Repeats important tests after fixes or changes to ensure existing features still work.
Usability Testing	Checks clarity, ease of navigation, labels, forms, feedback, and overall user experience.
Performance Testing	Checks loading, response time, repeated operations, and stability under normal usage.
Compatibility Testing	Checks supported browsers, devices, operating systems, and screen sizes.

Testing Level / Type	Purpose
Security Testing	Checks authentication, authorization, input handling, session behaviour, and protection of sensitive information.

4. Test Environment

Item	Configuration / Description
Application	FitBuddy fitness and wellness application.
Client	Supported web or mobile client according to the implemented project.
Database / Storage	Configured FitBuddy database or application storage layer.
Network	Normal internet connection plus controlled poor-network scenarios where applicable.
Devices	Representative desktop, laptop, tablet, and/or smartphone devices.
Browsers	Supported modern browsers relevant to the project implementation.
Test Data	Valid, invalid, empty, duplicate, boundary, and representative fitness-related data.

5. Functional Test Cases

ID	Module	Test Condition / Input	Expected Result
TC-01	Launch	Open FitBuddy.	Application loads successfully and displays the correct initial screen.
TC-02	Registration	Enter valid required registration information.	Account is created successfully and appropriate confirmation/navigation is shown.
TC-03	Registration	Leave required fields empty.	Submission is prevented and clear validation messages are displayed.
TC-04	Registration	Enter invalid email/phone/password format as applicable.	Invalid data is rejected with a meaningful validation message.
TC-05	Registration	Use an already registered identifier.	Duplicate registration is prevented and the user is informed.
TC-06	Login	Enter valid credentials.	User is authenticated and taken to the correct authenticated screen.
TC-07	Login	Enter incorrect credentials.	Access is denied and a safe error message is displayed.
TC-08	Logout	Select logout from an authenticated session.	Session ends and protected pages require authentication again.
TC-09	Profile	Edit profile using valid values and save.	Updated profile information is saved and displayed correctly.
TC-10	Profile	Enter invalid or incomplete profile data.	Invalid data is rejected and correction guidance is displayed.
TC-11	Workout	Create/select/start a supported workout.	Workout information is processed and displayed correctly.
TC-12	Workout	Use missing, invalid, or boundary workout values.	Validation or safe handling occurs according to requirements.
TC-13	Progress	Add/update fitness or progress information.	Data is stored and shown accurately.
TC-14	Persistence	Save data, reload or revisit the module.	Previously saved information remains available as expected.
TC-15	Navigation	Use menus, buttons, back controls, and links.	Each control opens the correct screen without broken navigation.
TC-16	Empty State	Open a feature with no available data.	A useful empty-state message or default view is shown.
TC-17	Error Handling	Trigger a controlled network/storage/service failure.	Application shows a safe error/retry state without exposing technical details.
TC-18	Search/Filter	Use valid and invalid search/filter values where applicable.	Correct matching results are displayed; no-result cases are handled clearly.

6. Boundary Value and Input Validation Testing

Test Area	Test Input	Expected Result
Minimum value	Enter the lowest permitted numeric value.	Accepted if it satisfies the requirement.
Below minimum	Enter one value below the permitted minimum.	Rejected with an appropriate validation message.
Maximum value	Enter the highest permitted numeric value.	Accepted if it satisfies the requirement.
Above maximum	Enter one value above the permitted maximum.	Rejected or safely constrained according to requirements.
Empty required field	Leave a required field blank.	Submission is prevented and the field is identified.
Optional field	Leave an optional field blank.	Application continues normally if the field is not required.
Special characters	Enter unexpected characters into text/numeric fields.	Input is validated and unsafe/unexpected content is handled safely.
Long input	Enter text longer than the supported limit.	Input is limited or rejected without breaking the interface.

7. Authentication and Authorization Testing

Test	Expected Result
Valid login	Correct credentials provide authenticated access.

Test	Expected Result
Invalid login	Incorrect credentials do not provide access.
Empty login fields	Validation messages are displayed.
Logout	The active session is terminated.
Protected page after logout	Protected content cannot be accessed without valid authentication.
Session handling	Expired or invalid sessions are handled safely.
Unauthorized action	A user cannot perform actions outside their permitted access.
Sensitive error messages	Errors do not unnecessarily reveal passwords, tokens, database details, or internal implementation information.

8. Non-Functional Testing

Category	Testing Activity	Expected Outcome
Performance	Measure screen/module loading and response during normal use.	Acceptable response time and no unnecessary delays.
Reliability	Repeat common operations and workflows.	Application remains stable without unexpected crashes.
Usability	Evaluate forms, labels, navigation, messages, and layout.	Users can understand and complete tasks with minimal confusion.
Responsiveness	Test different screen widths and orientations where applicable.	Content remains readable and controls remain usable.
Compatibility	Run application on supported browsers/devices.	Consistent behaviour across supported environments.
Security	Test authentication, authorization, validation, sessions, and error disclosure.	Unauthorized access is prevented and sensitive information is protected.
Maintainability	Review modular behaviour and regression impact after changes.	Changes can be made without unnecessary breakage of existing functionality.
Accessibility	Check readable text, labels, keyboard/focus behaviour, contrast, and usable controls where applicable.	Core functions remain accessible to intended users.

9. Integration Testing

Integration testing verifies that individual FitBuddy modules work correctly when connected. Examples include the relationship between login and the user dashboard, profile updates and stored user information, workout activities and progress records, and frontend forms and backend/database services.

Integration Scenario	Expected Result
Registration → Login	Newly registered user can authenticate using the stored credentials.
Login → Dashboard	Authenticated user reaches the correct dashboard and relevant data is loaded.
Profile → Storage	Profile changes are persisted and reflected on subsequent access.
Workout → Progress	Completed workout information updates the appropriate progress data where designed.
Frontend → Backend/API	Valid requests receive correct responses and errors are handled safely.
Database → UI	Stored records are retrieved accurately and displayed without unintended transformation.

10. Defect Reporting and Management

Whenever a test fails, the defect should be documented with enough information for a developer to reproduce and correct it. A typical defect record contains the defect ID, module, title, environment, preconditions, steps to reproduce, test data, expected result, actual result, severity, priority, screenshots/logs where appropriate, status, and retest result.

Status	Meaning
New	Defect has been identified and recorded.
Assigned	Defect has been assigned for investigation/fix.
In Progress	Developer is working on the defect.
Fixed	Developer reports that the defect has been corrected.
Retest	Tester verifies the correction using the original test case.
Reopened	The issue still occurs after the reported fix.
Closed	Fix has been verified and the defect no longer occurs.

11. Regression Testing

Regression testing is performed after bug fixes, database changes, UI changes, authentication updates, or addition of new FitBuddy features. Core workflows such as application launch, registration, login, logout, profile updates, workout operations, progress tracking, navigation, and data persistence should be re-tested after significant changes.

12. Test Result Summary

The final project report should record the actual execution status of each test case. The following format can be used for the FitBuddy test execution summary.

Metric	Value / Entry
Total Test Cases	To be filled after final execution
Passed	To be filled
Failed	To be filled
Blocked	To be filled
Not Executed	To be filled
Defects Identified	To be filled
Defects Fixed	To be filled
Regression Status	To be filled after retesting
Final Test Status	To be filled based on actual test evidence

13. Test Completion Criteria

Testing can be considered complete when the planned test cases have been executed, critical functionality has been verified, identified high-impact defects have been resolved or formally accepted, regression testing has been completed, and the application demonstrates stable behaviour in the supported environment.

14. Conclusion

The testing phase provides evidence that FitBuddy has been systematically evaluated across functional and non-functional areas. By testing normal user workflows as well as invalid inputs, boundary conditions, authentication, data persistence, navigation, error handling, compatibility, usability, performance, and regression scenarios, the project can reduce defects and improve overall reliability. Final acceptance should be based on the actual executed test results and documented defect status.

15. Recommended Test Evidence

1. Screenshots of successful and failed test cases.
2. Test case execution sheet with Pass/Fail status.
3. Defect reports and retest evidence.
4. Database/API response evidence where applicable.
5. Browser/device compatibility screenshots.
6. Performance observations or measured response times.
7. Final regression-test checklist.